

Plan de Pruebas Detallado para el Componente de Segmentación (Segmentación basada en semillas con Dijkstra)

1. Objetivo

Verificar la correcta implementación y funcionamiento del algoritmo de segmentación basado en componentes conectados usando semillas y algoritmo de Dijkstra, asegurando que:

- La segmentación produzca etiquetas correctas para las regiones.
- Se manejen adecuadamente casos límite y de error.
- Se mantenga la robustez y estabilidad del componente bajo diferentes condiciones de entrada.

2. Alcance

Pruebas funcionales y no funcionales del módulo `segmentar_imagen_dijkstra` y sus integraciones con la clase `Imagen` y la estructura `Seed`. Se prueba la entrada, proceso y salida de la segmentación en diferentes escenarios.

3. Elementos a Probar

- Validación de entrada (imagen válida, semillas válidas).
- Algoritmo principal de segmentación (Dijkstra adaptado).
- Asignación correcta de etiquetas a píxeles.
- Manejo de píxeles no asignados.
- Rendimiento y estabilidad para imágenes grandes.
- Integración con entrada/salida de imágenes y consola.

4. Criterios de Aceptación

- El método retorna `true` para casos válidos y `false` para entradas inválidas.
- La imagen resultante contiene etiquetas coherentes y correctamente asignadas.
- No se presentan errores de acceso a memoria o fallos inesperados.
- El tiempo de ejecución es razonable para imágenes pequeñas y medianas.
- El resultado visual (al guardar imagen segmentada) corresponde a regiones diferenciadas por etiquetas.

5. Plan de pruebas

ID	Caso de Prueba	Entrada	Acción Esperada	Resultado Esperado
CP1	Imagen válida pequeña, una semilla	Imagen 10x10 con valores en escala de grises, semilla en (5,5), etiqueta=1	Ejecutar segmentación con esa semilla	Imagen salida con una única región etiquetada correctamente, todos píxeles asignados a etiqueta 1
CP2	Imagen válida pequeña, múltiples semillas	Imagen 10x10, semillas en (2,2), (7,7), etiquetas 1 y 2	Ejecutar segmentación	Imagen salida con dos regiones etiquetadas, cada píxel asignado a la semilla más cercana por costo acumulado
CP3	Imagen vacía	Objeto Imagen con dimensiones 0x0	Ejecutar segmentación	Retorna false, no se modifica la imagen salida
CP4	Semillas vacías	Imagen válida 10x10, vector de semillas vacío	Ejecutar segmentación	Retorna false, no se modifica la imagen salida
CP5	Semillas fuera de rango	Imagen válida 10x10, semilla en (11,11), etiqueta 1	Ejecutar segmentación	Retorna false o ignora semillas inválidas; no asigna regiones incorrectas
CP6	Etiquetas inválidas	Imagen válida 10x10, semilla en (5,5), etiqueta 0 (fuera rango permitido 1-255)	Ejecutar segmentación	Retorna false o ignora etiquetas inválidas
CP7	Imagen con píxeles uniformes	Imagen 10x10 con todos valores 100, dos semillas	Ejecutar segmentación	Píxeles asignados a la semilla que los alcanzó primero, sin diferencia de costo
CP8	Imagen con gradiente de intensidad	Imagen 10x10 con valores en gradiente (0 a 255), varias semillas	Ejecutar segmentación	Regiones etiquetadas correctamente según menor costo acumulado
CP9	Imágenes grandes (ej. 256x256), varias semillas	Imagen 256x256 con valores aleatorios, 3 semillas	Ejecutar segmentación	Segmentación completada sin errores, tiempos razonables, regiones coherentes
CP10	Píxeles inaccesibles (ej. regiones sin semillas)	Imagen 10x10 con píxeles muy diferentes y semillas limitadas	Ejecutar segmentación	Píxeles no alcanzados etiquetados con valor por defecto (ej. 0)
CP11	Verificación de salida en formato PGM	Imagen válida, semillas válidas	Ejecutar segmentación y guardar imagen salida	Archivo .pgm válido, con etiquetas visibles como intensidades de píxeles
CP12	Robustez ante inputs no numéricos (consola)	Comando segmentar con parámetros no válidos (letras, valores fuera de rango)	Invocar consola interactiva	Mensajes de error claros, sin crasheos

6. Procedimiento General para Cada Caso

1. Preparar la imagen de entrada (crear o cargar).
2. Definir el vector de semillas con sus coordenadas y etiquetas.
3. Invocar `segmentar_imagen_dijkstra(imagen_entrada, semillas, imagen_salida)`.
4. Validar el valor de retorno (`true` o `false`).
5. Validar que `imagen_salida` tenga dimensiones correctas.
6. Validar etiquetas asignadas manualmente o visualmente.
7. (Opcional) Guardar imagen de salida en PGM y verificar archivo.
8. Registrar tiempos de ejecución para pruebas de rendimiento.
9. Verificar manejo adecuado de errores y mensajes.

7. Recursos Necesarios

- Imágenes de prueba (generadas o capturadas).
- Herramientas para visualizar archivos PGM.
- Herramientas para medir tiempo de ejecución.
- Consola interactiva para pruebas integradas.

8. Indicadores de Calidad

- Cobertura: Probar todas las rutas de código, condiciones y validaciones.
- Rendimiento: Tiempos adecuados para tamaños estándar.
- Robustez: No debe haber fallas o crashes ante inputs erróneos.
- Exactitud: Resultados de segmentación coherentes con algoritmo teórico.

9. Notas Adicionales

- Para las pruebas automáticas, puede implementarse un conjunto de imágenes pequeñas y semillas predefinidas con resultados esperados conocidos para comparación automatizada.
- La segmentación puede visualizarse coloreando las etiquetas de las regiones para facilitar la inspección.
- En casos límite (CP3-6), validar que el programa no realice asignaciones incorrectas ni falle abruptamente.